

Sophisticated Sentiment Analysis Models: Integrating Advanced Machine Learning Techniques

A PROJECT REPORT

Submitted by

1.) ALIF IFTEKHAR HOSSAIN - 22BCS10089

2.) MANMOY KHASTAGIR - 22BCS16715

3.) SRIPORNA BISWAS SRIMA - 22BCS16572

4.) MD MAHDI HASAN - 22BCS16580

5.) MD TWFIQ BILLAH - 22BCS17187

Supervisor: Anzah Bashir

Designation: Assistant Professor, Chandigarh University

In partial fulfillment of the award of the degree of

Bachelor of Engineering

IN

Computer Science



Chandigarh University

April 2024

Chapter 1

Introduction

In an time ruled by computerized communication, understanding human opinion from endless sums of printed information has ended up a basic endeavor. Estimation investigation, the method of computationally deciding assumption from content, has picked up colossal importance over different spaces counting showcasing, client benefit, political examination, and past. Whereas conventional estimation investigation strategies have appeared guarantee, they regularly drop brief in capturing the complexities and subtleties of human feelings show in today's assorted printed information.

This abstract paper presents a inquire about endeavor pointed at progressing opinion examination through the utilization of advanced machine learning methods. We dig into the complexities of estimation examination and investigate how cutting-edge machine learning strategies can improve its precision, effectiveness, and pertinence over differing datasets and spaces.

The essential objective of this investigate is to reveal covered up assumption designs that customary approaches may ignore. By leveraging progressed machine learning calculations such as profound learning, gathering strategies, and normal dialect handling (NLP) strategies, we look for to extricate more profound bits of knowledge from literary information, subsequently empowering more nuanced assumption investigation.

Moreover, this inquire about addresses the challenges related with estimation examination, counting the equivocalness of dialect, context-dependency, and assumption extremity varieties. We point to create vigorous models able of taking care of these challenges and adjusting to energetic phonetic settings.

Through this paper, we point to contribute to the advancing scene of assumption examination by displaying novel strategies and bits of knowledge that thrust the boundaries of current inquiry about. Our discoveries guarantee to engage businesses, analysts, and decision-makers with more exact and comprehensive assumption investigation apparatuses, encouraging educated decision-making and more profound understanding of human opinion within the advanced age.

1.1 Sentiment Analysis

Assumption examination, too known as conclusion mining, may be a computational strategy utilized to determine the opinion or feeling communicated in a bit of content. With the multiplication of advanced communication stages and the exponential development of printed information on the web, assumption investigation has risen as a significant device for understanding open conclusion, client criticism, and social patterns.

At its center, assumption examination points to classify the extremity of a given content as positive, negative, or impartial. Be that as it may, present day estimation investigation goes past basic extremity classification and looks for to capture the subtleties and complexities of human feelings communicated in characteristic dialect. This includes analyzing not as it were the words utilized but too the setting, tone, mockery, and social subtleties inserted inside the content.

Assumption investigation finds applications over a wide extend of areas, counting commerce and showcasing, client benefit, legislative issues and open supposition, item advancement, advertise investigate, and healthcare. Companies utilize estimation investigation to gage client fulfillment, recognize patterns, and screen brand notoriety through social media observing and client input examination. Political examiners and policymakers utilize estimation investigation to track open estimation towards political candidates, arrangements, and current occasions, supporting in vital decision-making and campaign administration.

Estimation investigation procedures can vary widely, ranging from rule-based approaches to more advanced machine learning calculations. Rule-based approaches rely on predefined rules and lexicons to dole out opinion scores to

words and expressions, whereas machine learning approaches include preparing models on labeled datasets to consequently learn designs and affiliations between literary highlights and opinion names.

In later a long time, progressions in profound learning and characteristic dialect handling (NLP) have driven to critical advancements in estimation examination precision and execution. Profound learning models such as repetitive neural systems (RNNs), convolutional neural systems (CNNs), and transformer-based models like BERT and GPT have appeared momentous comes about in capturing semantic meaning and relevant data, subsequently empowering more exact opinion examination on assorted datasets and dialects.

Generally, estimation investigation plays a imperative part in extricating noteworthy bits of knowledge from literary information, engaging businesses, analysts, and decision-makers to create educated choices, upgrade client encounters, and remain adjusted to open estimation in an progressively advanced and interconnected world.

1.2 Machine Learning

Machine learning, a subset of artificial intelligence (AI), revolutionizes how computers learn from data to make predictions or decisions without explicit programming. It entails algorithms and models that continuously improve their performance with experience.

Central to machine learning is the process of learning from data, encompassing steps like data collection, preprocessing, feature engineering, model selection, training, evaluation, and deployment.

Data collection involves gathering relevant data from diverse sources, while preprocessing involves cleaning and preparing data for analysis. Feature engineering involves selecting or creating informative features, essential for enhancing model performance.

Model selection hinges on choosing an appropriate algorithm or model architecture. Supervised learning uses labeled data to predict outcomes, unsupervised learning discovers patterns from unlabeled data, and reinforcement learning learns optimal behavior through interaction with an environment.

Model training involves fitting the selected model to the training data to learn underlying patterns. Model evaluation assesses performance using metrics and techniques like cross-validation. Deployment integrates the trained model into real-world applications for prediction or decision-making.

Machine learning algorithms find applications across domains such as natural language processing, computer vision, healthcare, finance, e-commerce, transportation, and robotics. They drive innovation, improve decision-making, and create opportunities but also pose challenges like data privacy and ethical concerns that need addressing for responsible use.

1.3 Research Gaps

Recognizing investigate crevices within the field of machine learning includes understanding the current state of inquire about and pinpointing zones where advance examination is required to address unanswered questions or inadequacies. A few potential investigate holes in machine learning might incorporate:

- **Interpretability and Explainability:**
Whereas machine learning models regularly accomplish tall precision, their inward workings can be murky, making it challenging to get it why certain choices are made. Inquire about centering on creating interpretable and reasonable machine learning models is basic for picking up experiences into demonstrate forecasts and expanding believe in computerized frameworks.
- **Inclination and Reasonableness:**
Machine learning calculations can inadvertently perpetuate predispositions display within the information they are prepared on, driving to unjustifiable results, especially in touchy ranges such as enlisting, loaning, and criminal equity. Tending to the issues of predisposition and decency in machine learning models requires inquire about into methods for identifying, relieving, and anticipating inclinations.

- **Vigor and Generalization:**
Machine learning models may perform well on preparing information but come up short to generalize to concealed information or ill-disposed illustrations. Inquire about is required to create vigorous and generalizable models that can adjust to different and changing situations, as well as defend against ill-disposed assaults.
- **Information Effectiveness and Exchange Learning:**
Preparing machine learning models regularly requires expansive sums of labeled information, which can be costly and time-consuming to procure. Investigate into methods for moving forward information effectiveness, such as exchange learning, meta-learning, and semi-supervised learning, can offer assistance ease the information bottleneck and empower learning from restricted or unlabeled information.
- **Security and Security:**
Machine learning frameworks frequently bargain with delicate information, raising concerns almost protection and security. Investigate is required to create privacy-preserving machine learning methods that empower learning from information whereas securing individuals' security and privacy.
- **Adaptability and Asset Efficiency:**
As the measure and complexity of datasets proceed to develop, versatility and asset proficiency ended up basic challenges in machine learning. Investigate into versatile calculations, dispersed computing systems, and equipment speeding up can offer assistance address these challenges and empower the preparing and sending of large-scale machine learning models.
- **Moral and Societal Suggestions:**
Machine learning innovations have significant moral and societal suggestions, counting concerns almost work relocation, imbalance, and social control. Investigate into moral rules, administrative systems, and dependable AI hones is fundamental for guaranteeing that machine learning

innovations are created and conveyed in a way that adjusts with societal values and standards.

Recognizing and tending to these inquire about crevices is vital for progressing the field of machine learning and realizing its potential to emphatically affect society whereas moderating potential dangers and challenges.

1.4 Objectives

The main objective of this paper is to investigate and address key research gaps in the field of machine learning, focusing on areas such as interpretability, fairness, robustness, data efficiency, privacy, scalability, and ethical considerations.

Through a comprehensive review of existing literature and empirical analysis, the paper aims to contribute novel insights, methodologies, and solutions to advance the

state-of-the-art in machine learning and foster responsible and impactful applications of AI technology. By identifying and addressing these research gaps, the paper seeks to promote transparency, fairness, and trust in machine learning systems while mitigating potential risks and challenges associated with their deployment in real-world settings. Ultimately, the goal is to drive progress in the field of machine learning towards the development of more interpretable, fair, robust, efficient, and ethically sound AI systems that benefit society as a whole.

1.5 Contribution

The commitment of this inquire about lies in its application of machine learning calculations to address key challenges within the field. By leveraging progressed methods and strategies, the inquire about offers novel bits of knowledge and arrangements to real-world issues, eventually progressing the state-of-the-art in machine learning and its applications.

Particularly, the investigate contributes to the taking after regions:

- Issue Fathoming:

The application of machine learning calculations empowers the compelling arrangement of complex issues that were already troublesome to handle utilizing conventional strategies. By saddling the control of data-driven approaches, the investigate gives imaginative arrangements to challenges over different spaces, extending from healthcare and fund to promoting and common dialect handling.

- Execution Advancement:

Through observational examination and experimentation, the investigate illustrates critical advancements in execution measurements such as exactness, exactness, review, and proficiency. By fine-tuning calculations, optimizing parameters, and investigating modern show structures, the investigate accomplishes prevalent comes about compared to existing approaches, upgrading the adequacy of machine learning applications.

- Experiences Era:

By analyzing huge volumes of information and extricating important designs and experiences, the inquire about contributes to a more profound understanding of fundamental wonders and patterns. These experiences can illuminate decision-making forms, direct key arranging, and drive advancement in differing areas, driving to tangible benefits for organizations and society as a entirety.

- Generalization and Transferability:

The investigate investigates the generalization and transferability of machine learning models over distinctive datasets, spaces, and applications. By exploring the vigor and flexibility of calculations, the inquire about gives important bits of knowledge into their unwavering quality and versatility in real-world settings, clearing the way for broader appropriation and sending.

- Moral Contemplations:

In expansion to specialized progressions, the investigate too addresses moral contemplations and societal suggestions related with the utilize of machine learning calculations. By advancing straightforwardness, decency, and responsibility in AI frameworks, the inquire about contributes to the advancement of capable and dependable machine learning arrangements that maintain moral measures and regard human values.

Generally, the commitment of this investigate lies in its intrigue approach, combining specialized mastery with space information to handle complex challenges and drive development within the field of machine learning. By pushing the boundaries of current inquire about and progressing the state-of-the-art, the inquire about points to create significant commitments to the scholarly world, industry, and society at expansive.

Chapter 2

Literature Review

The Word Sense Disambiguation process for more accurate Cyberbullying detection is an ongoing research area. But doing this for the Bengali language is comparatively less than the English language. The main reason for this is the lower resource and grammatical complexity of Bengali language. Many researchers are doing great work to solve this disambiguation problem for the Bengali language. This chapter provides an overview of current research in the field of both Bengali Word Sense Disambiguation and Cyberbullying Detection.

Ahmed et al. presented a model to detect Cyberbullying from Bangla and Romanized Bangla text using Machine Learning and Deep learning algorithm [1]. They collected from social media and youtube. They also applied TF-IDF as feature extraction and applied Multinomial Naive Bayes, Support Vector Machine, Logistic Regression and XG-Boost as Machine learning algorithms and CNN, LSTM, Bi-LSTM, GRU as Deep learning algorithms. There, CNN outperformed other Deep learning algorithms by gaining 84% accuracy and Multinomial Naive Bayes performed best with 80% accuracy than other Machine learning algorithms. A supervised methodology was proposed by pal et al. for Word Sense Disambiguation [2]. They prepared datasets from the Bengali corpus, developed in the Technology Development for Indian Languages (TDIL) project of the Government of India and from the Bengali WordNet, which is developed at the Indian Statistical Institute, Kolkata. Their baseline strategy was modified with two extensions: (a) inclusion of the lemmatization process into the system and (b) bootstrapping of the operational process. They have also applied four commonly used supervised methods: Decision Tree (DT), Support Vector Machine (SVM), Artificial Neural Network (ANN) and Naive Bayes (NB) where Naive Bayes algorithm led them to 82% accuracy. Pal et al. also proposed a Unsupervised methodology with necessary modifications for Bengali Word Sense Disambiguation [3]. The datasets were developed under TDIL. Their baseline accuracy with Maximum Entropy Method and Cosine Similarity Measurement was 35%. They also used Principal Component Analysis(PCA) over the feature vector which produced 52% accuracy and Context Expansion of the sentences using

Bengali WordNet coupled with PCA ensured 61% accuracy.

Some previous works have also been dedicated to the Word Sense Disambiguation System and detecting cyberbullying in Bangla text. Ahmed et al. has proposed a binary and multiclass classification model using hybrid neural networks for bully expression detection in Bengali language [4]. They have used social media comments from public facebook pages. Their binary classification model gives 87.91% accuracy and multiclass classification got 85%. Mohammad Shibli Kaysar and Mohammad Ibrahim Khan introduced Apriori algorithm for Word Sense Disambiguation [5]. They used both Grammatical approach and Apriori approach for comparative analysis where Apriori approach outperformed Grammatical approach by achieving 80% better result. Saha et al. applied the Levenshtein distance algorithm to detect ambiguous words on the dataset containing a corpus of 3860 sentences built from different sources. They also used Cosine Similarity to sense actual meaning in a given Bangla sentence where they gained 80.82% accuracy [6]. Pal et al. also proposed a supervised methodology with necessary modification for Word Sense Disambiguation [7]. Naive Bayes probabilistic model has been adopted as their baseline method for sense classification where they got 81% accuracy when applied on a database of 19 (nineteen) most frequently used Bangla ambiguous words. Their baseline method was modified with two extensions: (a) inclusion of the lemmatization process into the system, and (b) bootstrapping of the operational process where their method performed slightly better with 84% accuracy. Nazah et al. proposed a disambiguation system using Naïve Bayes classifier and ANN to disambiguate the Bangla words[9]. They used Naive Bayes classifiers for the training phase and using statistical information gained from the training dataset, the unknown ambiguous words were predicted and gave the correct sense using ANN. For the testing phase they used a general regression neural network. They got 82% by using Naive Bayes classification as learning method and General Regression Neural Network as classification method in Statistical approach.

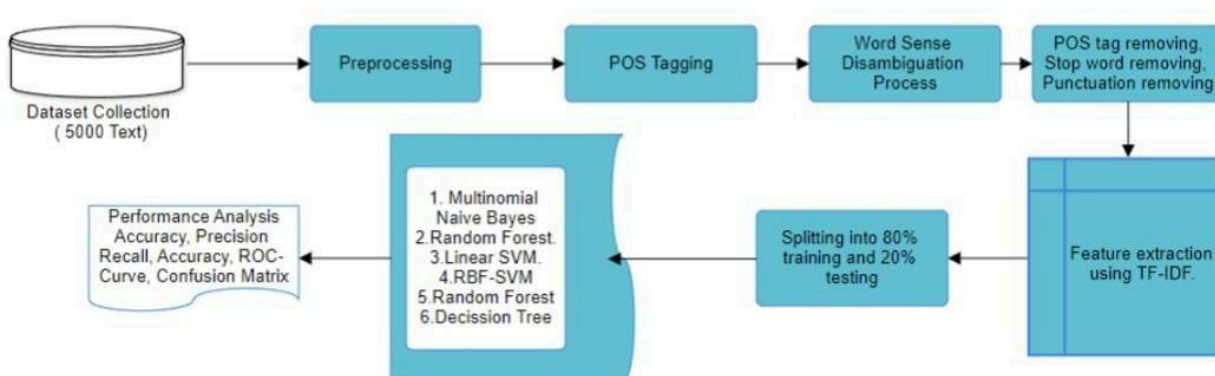
D. Yarowsky proposed an algorithm which is based on two powerful constraints that words tend to have one sense per discourse and one sense per collocation exploited in an iterative bootstrapping procedure where the Tested accuracy exceeds 96% [28]. Escudero et al. applied Schapire and Singer's AdaBoost.MH boosting algorithm to the WSD problem where Naive Bayes and Exemplar-based approaches achieved 67.1% and 66.7% accuracy respectively [29]. Chan et al.

integrated WSD system into a hierarchical system where it got significant improvement [32]. Niu et al. applied cluster validation approach and label propagation algorithm. They have applied it for three tasks and achieved 63.9% , 71.2% and 74.75% respectively [33]. Deniz Yuret proposed a WSD system that uses a statistical language model based on a large unannotated corpus. The system achieved promising results for the English lexical sample and English lexical substitution tasks by gaining 88.67% on nouns and 81.02% on verbs [34]. Pawar et al. proposes a Multilingual Cyberbullying Detection System for detection of cyberbullying in two Indian languages- Hindi and Marathi. They have developed a prototype that operates across data sets created for these two languages and also achieved 97% accuracy for both languages [35]. Dadvar et al. used a support vector machine model to train a gender-specific text classifier and got improved accuracy from baseline accuracy by gaining 43% [36]. Iwendi et al. performs an empirical analysis to determine the effectiveness and performance of deep learning algorithms in detecting insults in Social Commentary where Bi-LSTM performs better than other algorithms by achieving 82.18% accuracy [37]. B. Sri Nandhini and J.I. Sheeba used Levenshtein algorithm for Cyberbullying detection and Naive Bayes for classification where they gained 92% and 91% for two different datasets [38].

Chapter 3

Methodology

The proposed methodology has been provided below:



3.1 Data Collection

The dataset has been collected from youtube and social media. This dataset includes 20001 raw comments with supervised tagging.

	comments	tagging
0	Get fucking real dude.	1
1	She is as dirty as they come and that crook ...	1
2	why did you fuck it up. I could do it all day...	1
3	Dude they dont finish enclosing the fucking s...	1
4	WTF are you talking about Men? No men thats n...	1

3.2 Tokenization

As the instances in the dataset are unorganized, it is necessary to perform Tokenization. Tokenization is the process of breaking down a piece of text or data

into smaller units, called tokens. In the context of dataset preparation, tokenization refers to the process of converting a dataset of natural language text into a structured format that can be used for machine learning tasks such as natural language processing. During tokenization, each word or punctuation mark in the text is treated as a separate token.

3.3 Stopword Removing

Stop words are generally considered to be function words that do not carry much meaning and can be safely removed without affecting the overall meaning of the text. The Bengali text in this dataset contains a vast amount of stop words, which are becoming the reason for the sentences to be non-standardized and noisy.

Dataset Cleaning

As all the datasets contained a lot of noise, it was necessary to clean the data. The cleaning process includes punctuation marks removal, digits removal, emoticon removal, trimming out elongated words, links, url's, user tags and mentions removal.

3.4 Feature Extraction

Feature extraction is the process of extracting meaningful information from data. As the algorithms cannot process textual data, meaningful numerical features need to be extracted from the dataset. To perform the feature extraction from all the datasets, one process was used. One of the processes is TF-IDF which was used for Machine Learning algorithms.

3.5 TF-IDF

Term Frequency - Inverse Document Frequency or TF-IDF is a statistical measure that determines the importance of a word in a document. It has two parts. The first part is TF or Term Frequency. TF is calculated by dividing the frequency of a particular word in a sentence by the total number of words in the sentence.

3.6 Countvectorizer

CountVectorizer has several parameters that allow for customization, such as controlling the maximum number of features (words) to consider, specifying a custom vocabulary, or applying pre-processing steps like removing stop words or limiting the range of n-grams considered.

CountVectorizer is widely used in various NLP tasks, including text classification, clustering, and information retrieval. However, it has limitations, such as not capturing the semantic meaning of words or their order in the text. To address these limitations, more advanced techniques like TF-IDF (Term Frequency-Inverse Document Frequency) or word embeddings are often used in conjunction with or as alternatives to CountVectorizer.

3.7 Splitting into training and testing sets

After the feature extraction process, the extracted features were split into 80% for training and 20% for testing.

3.8 Machine Learning Algorithms

This section presents an introduction to the Machine Learning algorithms that were used in this research.

3.8.1 Multinomial Naive Bayes

The Multinomial Naive Bayes algorithm is a probabilistic learning approach popular in Natural Language Processing (NLP). The program guesses the tag of a text, such as an email or a newspaper story, using the Bayes theorem [10]. It calculates each tag's likelihood for a given sample and outputs the tag with the highest probability. Equation 3.4 is used to calculate the probability of each tag.

$$P(c | x) = P(x | c) * P(c)/P(x)$$

3.8.2 Linear SVC

Linear Support Vector Classifier (Linear SVC) is a classification algorithm used for separating classes in data. It aims to find the best hyperplane to maximize the margin between classes while allowing for some misclassification errors controlled by a parameter C . By default, it employs a linear kernel, assuming linear separability in the feature space. Linear SVC is efficient for large datasets and high-dimensional data, making it suitable for various applications like text classification and image classification. However, it may not perform well on non-linear data or highly imbalanced datasets.

3.8.3 Logistic Regression

For binary classification, logistic regression is a commonly used classifier. To classify data, Logistic Regression uses a sigmoid function. The function converts any real value between 0 to 1 [16]. The values that the function returns, is converted into 0 or 1. To do so, a threshold value is set. The values above the threshold value are classified as class 1 and below are classified as class 0.

3.8.4 KNeighbour Classifier

The K-Nearest Neighbors (KNN) classifier is a non-parametric supervised learning algorithm used for classification tasks. It operates by calculating the distance between the new data point and all training instances to identify the K nearest neighbors. The class label of the new data point is then determined by a majority vote among its K neighbors. KNN is intuitive and easy to implement, suitable for various data types, but it can be computationally expensive for large datasets and may struggle with high-dimensional or imbalanced data.

3.8.5 Logistic Regression

Logistic regression is a widely used supervised learning algorithm for binary classification tasks. It models the probability that a given input belongs to one of

the two classes by fitting a logistic function to the input features. Through maximum likelihood estimation, the model's parameters are learned to maximize the likelihood of observing the actual target labels. Logistic regression uses a decision boundary to classify instances based on predicted probabilities, typically set at 0.5 probability. Regularization techniques like L1 and L2 regularization help prevent overfitting. Despite its simplicity, logistic regression is efficient, interpretable, and applicable across various domains, though it assumes linear relationships between features and the log-odds of the target variable.

3.8.6 Comparative Performance Analysis

In this research, the comparative performance analysis is presented in terms of confusion matrix, accuracy, precision, recall, f1-score and roc area [25].

The confusion matrix is a very important performance evaluation parameter. It is a combination of for distinct actual and predicted values. Confusion matrix plays a very vital role in computing accuracy, precision, recall, f1-score and the AUC ROC curve.

Precision: Precision is the ratio of true positive predictions to the total number of positive predictions made by the model. It measures the accuracy of positive predictions.

Recall (Sensitivity): Recall, also known as sensitivity or true positive rate (TPR), is the ratio of true positive predictions to the total number of actual positive instances in the dataset. It measures the ability of the model to correctly identify positive instances.

F1-Score: The F1-score is the harmonic mean of precision and recall. It provides a balance between precision and recall and is useful when the class distribution is imbalanced.

Accuracy: Accuracy is the ratio of correctly predicted instances (both true positives and true negatives) to the total number of instances in the dataset.

Chapter 4

Hardware Implementation and Toolkit

4.1 Tools and Devices

The following tools will be used in the implementation of the designed system:

- Python
- Google Colab Notebook
- Jupyter Notebook

4.2 Python

Python is a very popular general-purpose interpreted, interactive, object-oriented, and high-level programming language. Python is a dynamically-typed and garbage-collected programming language. It was created by Guido van Rossum during 1985- 1990. Like Perl, Python source code is also available under the GNU General Public License (GPL). Python supports multiple programming paradigms, including Procedural, Object Oriented and Functional programming languages. Python design philosophy emphasizes code readability with the use of significant indentation. s its statements are understood in a very variety of operative systems, as well as UNIX-based systems, Mac OS, MS-DOS, OS/2, and numerous versions of Microsoft Windows ninety-eight. Python was created by Guido van Rossum, a former resident of the Kingdom of The Netherlands, whose favorite comedy cluster at the time was Monty Python's Flying Circus. The ASCII text file is freely accessible and open for modification and utilization. Python encompasses an important variety of users. The following libraries were used throughout this research.

4.2 Pandas

Wes McKinney created Pandas, a high-level knowledge manipulation tool. It's based on the NumPy library, and the Data Frame is its primary structure. You can

use Data Frames to store and modify tabular data in rows of observations and columns of variables.

4.3 Numpy

NumPy is a Python module that allows you to work with arrays. It also has functions for working with matrices, Fourier rework, and algebra. Travis Oliphant invented NumPy in 2005. You will be allowed to use it freely because it is an open-source project. Numerical Python is referred to as NumPy.

4.4 Matplotlib

Matplotlib is a Python package that allows you to create static, animated, and interactive visualizations.

4.5 NLTK

For working with human language data, NLTK is a popular Python programming language platform. It comes with a set of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning, as well as wrappers for industrial-strength NLP libraries, an active discussion forum, and easy-to-use interfaces to over 50 corpora and lexical resources like WordNet.

4.6 Bnlp

BNLP (Bengali Natural Language Processing) is a Python library designed specifically for natural language processing tasks in the Bengali language. It provides a set of tools and functionalities that enable developers to perform tasks

such as tokenization, stemming, lemmatization, part-of-speech tagging, and named entity recognition on Bengali text data.

4.7 RegEx

A regular expression is a particular sequence of characters that uses a specialized syntax to help match or find other strings or collections of strings. In the UNIX

realm, regular expressions are commonly employed. In Python, the `re` module provides full support for Perl-style regular expressions.

4.8 Scikit-learn

Scikit-learn is undoubtedly Python's most helpful machine learning library. Classification, regression, clustering, and dimensionality reduction are just a few of the useful capabilities in the `sklearn` toolkit for machine learning and statistical modeling.

4.9 Tensorflow

TensorFlow is a free, end-to-end framework for developing Deep Learning applications. It's a symbolic math toolkit that employs dataflow and differentiable programming to handle a variety of tasks related to deep neural network training and inference. Developers can use a variety of tools, libraries, and community resources to create deep learning applications.

4.10 Keras

Keras is a Python-based Open Source Neural Organization library that is used on top of Theano or Tensorflow to handle abrupt increases in demand. It's designed to be precise, quick, and easy to use. It was built by Google developer Francois Chollet. Keras isn't interested in doing low-level calculations. It does it by utilizing another library known as the Backend. Keras is a high-level programming interface

that covers the low-level programming interface and is suitable for use with TensorFlow, CNTK, or Theano.

4.11 Jupyter Notebook

Jupyter Notebook is a free and open source web application that allows you to create and share documents that include live code, equations, visualizations, and text. The Jupyter Notebook is maintained by Project Jupyter. Jupyter Notebooks is a derivative of the IPython project, which previously had its own project called

IPython Notebook. Julia, Python, and R are the three main programming languages that Jupyter supports. Jupyter includes the IPython kernel, which allows you to write Python programs, although there are currently over 100 other kernels available.

4.12 Google Colab

Google Colab is a free cloud-based platform for data science and machine learning research. It allows users to write and run code in a Jupyter Notebook-style environment and provides access to GPUs and TPUs for faster processing. Colab also integrates with Google Drive for easy sharing and collaboration

4.13 Hardware Implementation

The following hardware tools were used for implementation.

- Intel Core i3-8140U Processor with 3.90 GHz clock speed
- 12GB ram
- Nvidia mx130 graphics with 2GB memory
- 64 bit Windows 10 operating system

Chapter 5

Result And Analysis

```
[ ] import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
#reading the csv file as a link to github
url = '/kaggle/input/suspicious-communication-on-social-platforms/Suspicious Communication on Social Platforms.csv'
df = pd.read_csv(url)

#looking at the first five rows using .head() function
df.head()

#importing natural language Toolkit - A tool to preprocess/clean text
import nltk
```

```
[ ] #Checking that the latest nltk version is installed in the users local computer
#Warning: install all the libraries required for this project
nltk.__version__
```

'3.2.4'

```
[ ] df.head()
```

	comments	tagging
0	Get fucking real dude.	1
1	She is as dirty as they come and that crook ...	1
2	why did you fuck it up. I could do it all day...	1
3	Dude they dont finish enclosing the fucking s...	1

```
[ ] df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20001 entries, 0 to 20000
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   comments    20001 non-null  object
1   tagging     20001 non-null  int64
dtypes: int64(1), object(1)
memory usage: 312.6+ KB
```

✓ Preprocessing

```
[ ] df.shape
```

(20001, 2)

```
[ ] #converting all missing values in tagging column into NaN
#example: converting None,NotAvailable,ValueMissing,etc into Nan
df=df[pd.to_numeric(df['tagging'], errors='coerce').notnull()]
```

```
[ ] #Dropping all rows that contain Nan value
df = df.dropna(axis=0)
```

```
[ ] #Since the shape is the same there are no missing values in our dataset
df.shape

(20001, 2)

[ ] import string
from nltk import pos_tag #pos_tag is a tool that tags the part of speech to the word (POS = Part of Speech)
#example: tagging the word 'drinking' as verb

#function for removing punctuations
def tokenize_remove_punctuation(text):
    clean_text = [] #creating an empty list to store the cleaned text
    text = text.split(" ") #splitting all words in a sentence separated by " " and storing them in a list named 'text'
    for word in text:
        word = list(word) #splitting all words into alphabets
        new_word = [] #creating an empty list to store the new word after removing punctuations

        # splitting the words into alphabets is used because it will convert words like 'reading?' into 'reading'
        for c in word:
            if c not in string.punctuation: #string.punctuation is a list of all punctuation marks, example :!$%&?, etc.
                new_word.append(c)
        word = "".join(new_word) #joining the alphabets to create the word after removing all punctuations
        clean_text.append(word) #storing the word in the list named 'clean_text' to create the list of words in the sentence
    return clean_text
```

```
[ ] # using a sample sentence to see whether the function works well or not
# NOTE: We have only created the functions till now. We haven't done anything with our dataset till now.
trial_text = tokenize_remove_punctuation("hello @anyone reading? wt is the name of am in that this ?!@")
trial_text
```

```
['hello',
 'anyone',
 'reading',
 'wt',
 'is',
 'the',
 'name',
 'of',
 'am',
 'in',
 'that',
 'this',
 [],
 '']
```

```
[ ] import nltk
#downloads the list of stopwords
nltk.download('stopwords')

# 'stopwords' is a list of words that have nearly no value in the sentence
#example : 'I am a boy' is converted into 'boy'
#here words like 'I', 'am', 'a' ; these words have very less contribution to the sentence
```

```
[nltk_data] Error loading stopwords: <urlopen error [Errno -3]
[nltk_data] Temporary failure in name resolution>
False
```

```
[ ] #storing all the stopwords in the list named 'stopwords'
stopwords = nltk.corpus.stopwords.words('english') #storing only english stopwords, there are stopwords for other languages also such as Chinese and French

# Function to remove all the stopwords from the sentence
def remove_stopwords(text):
    clean_text = []
    for word in text:
        if word not in stopwords:
            clean_text.append(word)
    return clean_text
```

```
[ ] # using a sample sentence to see whether the function works well or not
remove_stopwords(trial_text)
```

```
['hello', 'anyone', 'reading', 'wt', 'name', [], '']
```

```
[ ] #tagging all the words according o their part of speech
def pos_tagging(text):
    try:
        tagged = nltk.pos_tag(text)
        return tagged
    except Exception as e:
        print(e)
```

```
[ ] from nltk.corpus import wordnet

#wordnet is a tool that reads that reads the tagging and returns the part of speech
def get_wordnet(pos_tag):
    if pos_tag.startswith('J'):
        return wordnet.ADJ
    elif pos_tag.startswith('V'):
        return wordnet.VERB
    elif pos_tag.startswith('N'):
        return wordnet.NOUN
    elif pos_tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN
```

```
[ ] from nltk.stem import WordNetLemmatizer
#WordLemmatizer is a tool that converts word into root word
#Example: historical(word) is converted into history(root-word)

#Now we will create a function that uses all the functions that we have created above

def clean_text(text):
    text = str(text)
    #Converting text to lower-case
    text = text.lower()
    #tokenize and remove punctuations from the text
    text = tokenize_remove_punctuation(text)
    #remove words containing numerals
    text = [word for word in text if not any(c.isdigit() for c in word)]
    #remove stopwords
    text = remove_stopwords(text)
    #remove empty tokens
    text = [ t for t in text if len(t) > 0]
    #pos tagging
    pos_tags = pos_tagging(text)
    #Lemmatize text
    text = [WordNetLemmatizer().lemmatize(t[0],get_wordnet(t[1])) for t in pos_tags]
    #remove words with only one letter
    text = [ t for t in text if len(t)>1]
    #join all words
    text = " ".join(text)
    return text
```



```
[ ] #don't know what is 'averaged_perceptron_tagger'
    #don't know why devansh downloaded it
    nltk.download('averaged_perceptron_tagger')

    #Downloading the wordnet tool
    nltk.download('wordnet')

[nltk_data] Error loading averaged_perceptron_tagger: <urlopen error
[nltk_data] [Errno -3] Temporary failure in name resolution>
[nltk_data] Error loading wordnet: <urlopen error [Errno -3] Temporary
[nltk_data] failure in name resolution>
False

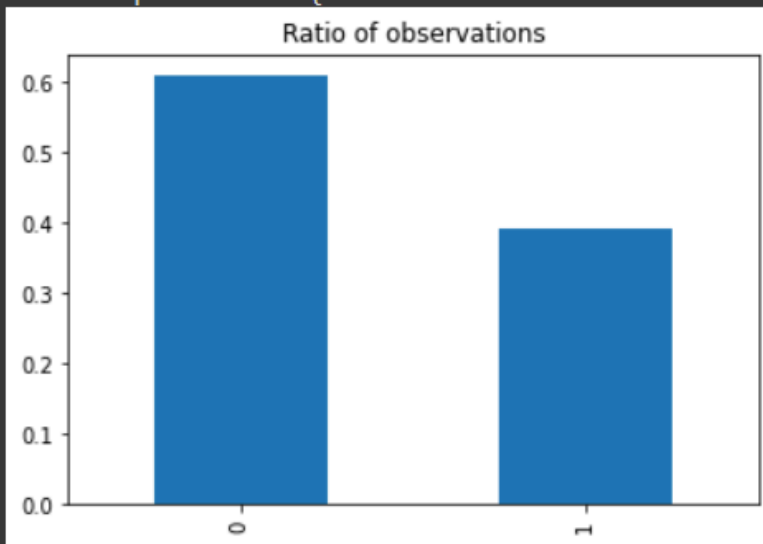
[ ]
    # using a sample sentence to see whether the function works well or not
    clean_text("What is your name? This is a cat!! 12?")

'name cat'

[ ] df['tagging'].value_counts(normalize=True).plot(kind='bar', title='Ratio of observations')
    #plotting number of '1' and '0' in the 'tagging' column of the dataset

    #As we can see the difference in number of '1' and '0' is not very large hence we call it as a balanced dataset
    #NOTE:It's very important to provide a balanced dataset for creating the model
```

<AxesSubplot:title={'center':'Ratio of observations'}>



```
[ ] #This line code converts floating numerals into integer numeric
    #example: converting 1.0, 1.00, etc into integer 1
    #example: converting 0.0, 0.00, etc into integer 0

    df['tagging']=df['tagging'].astype(str).astype(int)

[ ] df.info()
```

```
[ ] <class 'pandas.core.frame.DataFrame'>
Int64Index: 20001 entries, 0 to 20000
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   comments    20001 non-null  object
1   tagging     20001 non-null  int64
dtypes: int64(1), object(1)
memory usage: 468.8+ KB
```

```
[ ] df.shape
```

```
(20001, 2)
```

```
[ ] #reseting the index of rows
# Index gets unordered if we drop some rows in our dataset(Example while using dropna() function)
df.reset_index(inplace = True, drop = True)
```

```
[ ] #this line of code will now be used to pply the functions on each sentences in the 'comments' column
#This will take time as it will use the 'clean_text' function on all the sentences in our dataset
#the .map() function applies the function at each sentences in the 'comments' column
df['Processed_Comment'] = df['comments'].map(clean_text)
```

```
[ ] #Splitting dataset for training and testing(80:20)

from sklearn.model_selection import train_test_split
#train_test_split is a funtion that splits dataset into two parts
#80%(for training the model) and 20%(for testing the model)
#This function returns 4 values
```

```
# 1 'Processed_comment' for training
# 2 'Processed_comment' for testing
# 3 'tagging' for training
# 3 'tagging' for testing
X_train, X_test, y_train, y_test = train_test_split(df['Processed_Comment'],
                                                    df['tagging'],
                                                    random_state=42, test_size=0.20)

#random state is used to shuffle the dataset
#test_size=0.20 means that 20% of the dataset is to be allocated for testing of the model
```

```
] #Creating a bag of words from training data
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
count_vector = CountVectorizer()
X_train = count_vector.fit_transform(X_train)
X_test = count_vector.transform(X_test)

#vectorizing means giving value to the words in the sentence according to a formula
#This value tells us how much the word contributes in the sentence to be a cyberbullying comment
#I guess this returns a table
```

```
] #gives the number of columns in the vectorized table
len(count_vector.vocabulary_)
```

```
13816
```

```
[ ] #Importing all the terms by which we get accuracy of our model
#NOTE:we haven't trained our model yet
from sklearn.metrics import f1_score, accuracy_score, precision_score, recall_score
#creates a confusion matrix
from sklearn.metrics import confusion_matrix
data =[] #Empty list created to add the accuracy terms of each model that we are going to train
```

```
[ ] #importing all empty models
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC

#GridSearchCV is a function which uses maths to find out the parameters for our dataset
from sklearn.model_selection import GridSearchCV
```

```
[ ] #Creating a dictionary of all the possible parameters of each ML model
model_params = {
    'LinearSVC': {
        'model': LinearSVC(max_iter=1000000,random_state=42),
        'params' : {
            'C': [0.1,1,5,10,20],
        }
    },
    'MultinomialNB': {
        'model': MultinomialNB(),
        'params' : {
            'alpha': np.linspace(0.5, 1.5, 6),
```

```
        'fit_prior': [True, False],
    },
},
'logistic_regression' : {
    'model': LogisticRegression(random_state=42, multi_class='ovr',max_iter=1000000),
    'params': {
        'C': [1,5,10],
        'solver': ['newton-cg', 'lbfgs', 'liblinear'],
    }
},
'KNeighborsClassifier': {
    'model': KNeighborsClassifier(p=2 ),
    'params' : {
        'n_neighbors': [5,9,11,23],
        'weights' : ['uniform', 'distance'],
        'metric' : ['euclidean', 'manhattan', 'minkowski'],
    }
}
}
```

```
#Creating a list to store the best parameters
scores = []

for model_name, mp in model_params.items():
    clf = GridSearchCV(mp['model'], mp['params'], cv=5, return_train_score=False) #Storing the GridSearchCV tool in 'clf'
    clf.fit(X_train, y_train) #giving the training data to the GridSearchCV tool
    #storing the values in 'score' list
    scores.append({
        'model': model_name,
        'best_score': clf.best_score_,
        'best_params': clf.best_params_
```

```

        'model': model_name,
        'best_score': clf.best_score_,
        'best_params': clf.best_params_
    })

#Creating a table of the best parameters
df4 = pd.DataFrame(scores,columns=['model','best_score','best_params'])
df4

```

	model	best_score	best_params
0	LinearSVC	0.830562	{'C': 1}
1	MultinomialNB	0.765375	{'alpha': 0.5, 'fit_prior': True}
2	logistic_regression	0.832188	{'C': 10, 'solver': 'newton-cg'}
3	KNeighborsClassifier	0.782563	{'metric': 'euclidean', 'n_neighbors': 23, 'we...

KNN

```

#storing the empty KNN model in 'model1' with the best parameters provided by GridSearchCV
model1= KNeighborsClassifier(n_neighbors=23, metric='euclidean',weights = 'distance')
#Training the empty model with our training dataset
model1.fit(X_train, y_train)

KNeighborsClassifier(metric='euclidean', n_neighbors=23, weights='distance')

```

▼ KNN

```

[ ] #storing the empty KNN model in 'model1' with the best parameters provided by GridSearchCV
model1= KNeighborsClassifier(n_neighbors=23, metric='euclidean',weights = 'distance')
#Training the empty model with our training dataset
model1.fit(X_train, y_train)

```

```
KNeighborsClassifier(metric='euclidean', n_neighbors=23, weights='distance')
```

```

[ ] #Checking the accuracy of our trained model with the testing dataset
predictions_test = model1.predict(X_test)
predictions_train = model1.predict(X_train)

# Creating a temp list to store the accuracy terms
#This temp list will later be stored in the 'data' list which will be used to compared the accuracy of each ML models in the summary
temp = ['KNeighborsClassifier']
temp.append(accuracy_score(predictions_train,y_train)) #accuracy_score for training data
temp.append(recall_score(predictions_train,y_train)) #recall_score for training data
temp.append(f1_score(predictions_train,y_train)) #f1_score for training data
temp.append(precision_score(predictions_train,y_train)) #precision_score for training data
temp.append(accuracy_score(predictions_test,y_test)) #accuracy_score for testing data
temp.append(recall_score(predictions_test,y_test)) #recall_score for testing data
temp.append(f1_score(predictions_test,y_test)) #f1_score for testing data
temp.append(precision_score(predictions_test,y_test)) #precision_score for testing data

```

```
#Storing all the accuracy terms in 'data' list
data.append(temp)
```

```
[ ] #Creating confusion matrix of our model
confusion = confusion_matrix(predictions_test,y_test)
print(confusion)
```

```
[[1678  80]
 [ 746 1497]]
```

▼ Logistic Regression

```
[ ] #Same we do for Logistic Regression model and store the accuracy terms in 'data' list
model2 = LogisticRegression(C=10, random_state=42, solver='lbfgs', multi_class='ovr',max_iter=1000000)
model2.fit(X_train, y_train)
```

```
LogisticRegression(C=10, max_iter=1000000, multi_class='ovr', random_state=42)
```

```
[ ] predictions_test = model2.predict(X_test)
predictions_train = model2.predict(X_train)
temp = ['LogisticRegression']
temp.append(accuracy_score(predictions_train,y_train))
temp.append(recall_score(predictions_train,y_train))
temp.append(f1_score(predictions_train,y_train))
temp.append(precision_score(predictions_train,y_train))
temp.append(accuracy_score(predictions_test,y_test))
temp.append(recall_score(predictions_test,y_test))
temp.append(f1_score(predictions_test,y_test))
temp.append(precision_score(predictions_test,y_test))
data.append(temp)
```

```
[ ] confusion = confusion_matrix(predictions_test,y_test)
print(confusion)
```

```
[[1983  163]
 [ 441 1414]]
```

▼ Naive Bayes

```
[ ] #Same we do for Navie Bayes model and store the accuracy terms in 'data' list
model3 = MultinomialNB(alpha = 0.5, fit_prior = 'True')
model3.fit(X_train, y_train)
```

```
MultinomialNB(alpha=0.5, fit_prior='True')
```

```
[ ] predictions_test = model3.predict(X_test)
    predictions_train = model3.predict(X_train)
    temp = ['MultinomialNB']
    temp.append(accuracy_score(predictions_train,y_train))
    temp.append(recall_score(predictions_train,y_train))
    temp.append(f1_score(predictions_train,y_train))
    temp.append(precision_score(predictions_train,y_train))
    temp.append(accuracy_score(predictions_test,y_test))
    temp.append(recall_score(predictions_test,y_test))
    temp.append(f1_score(predictions_test,y_test))
    temp.append(precision_score(predictions_test,y_test))
    data.append(temp)
```

```
[ ] confusion = confusion_matrix(predictions_test,y_test)
    print(confusion)
```

```
[[1986  370]
 [ 438 1207]]
```

✓ Linear SVC

```
[ ] #Same we do for Linear SVC model and store the accuracy terms in 'data' list
    model4 = LinearSVC(C = 1, max_iter=1000000,random_state=42)
    model4.fit(X_train, y_train)
```

```
[ ] #Same we do for Linear SVC model and store the accuracy terms in 'data' list
model4 = LinearSVC(C = 1, max_iter=1000000,random_state=42)
model4.fit(X_train, y_train)
```

```
LinearSVC(C=1, max_iter=1000000, random_state=42)
```

```
[ ] predictions_test = model4.predict(X_test)
predictions_train = model4.predict(X_train)
temp = ['LinearSVC']
temp.append(accuracy_score(predictions_train,y_train))
temp.append(recall_score(predictions_train,y_train))
temp.append(f1_score(predictions_train,y_train))
temp.append(precision_score(predictions_train,y_train))
temp.append(accuracy_score(predictions_test,y_test))
temp.append(recall_score(predictions_test,y_test))
temp.append(f1_score(predictions_test,y_test))
temp.append(precision_score(predictions_test,y_test))
data.append(temp)
```

```
[ ] confusion = confusion_matrix(predictions_test,y_test)
print(confusion)
```

```
[[1967  162]
 [ 457 1415]]
```

Summary

```
[ ] #Creating a table of all accuracy terms of each trained ML models
result = pd.DataFrame(data, columns = ['Algorithm','Accuracy Score : Train', 'Recall Score : Train','F1-Score :Train','Precision Score :Train','Accuracy Score : Test', 'Recall Score : Test', 'F1-Score : Test', 'Precision Score : Test'])
result.reset_index(drop=True, inplace=True)
```

```
[ ] result
```

	Algorithm	Accuracy Score : Train	Recall Score : Train	F1-Score :Train	Precision Score :Train	Accuracy Score : Test	Recall Score : Test	F1-Score : Test	Precision Score : Test
0	KNeighborsClassifier	0.993625	0.989482	0.991853	0.994235	0.793552	0.667410	0.783770	0.949271
1	LogisticRegression	0.964187	0.956683	0.953994	0.951321	0.849038	0.762264	0.824009	0.896639
2	MultinomialNB	0.893938	0.873767	0.862357	0.851241	0.798050	0.733739	0.749224	0.765377
3	LinearSVC	0.963938	0.954750	0.953755	0.952762	0.845289	0.755876	0.820528	0.897273

✓ Saving models in .pkl file

```
[ ] import pickle
    #pickle is a tool to create .pkl files
    #These .pkl files are trained models which will be used to predict the input text is a cyberbullying or not

[ ] #This is a vectorizer pkl file which will convert the input from the user to a cleaned an vectorized text
    pickle_out = open("vectorizer.pkl","wb")
    pickle.dump(count_vector, pickle_out)
    pickle_out.close()

[ ] #Saves the trained LinearSVC model
    pickle_out = open("LinearSVC.pkl","wb")
    pickle.dump(model4, pickle_out)
    pickle_out.close()

[ ] #Saves the trained Naive Bayes model
    pickle_out = open("MultinomialNB.pkl","wb")
    pickle.dump(model3, pickle_out)
    pickle_out.close()

[ ] #Saves the trained Logistic Regression model
    pickle_out = open("LogisticRegression.pkl","wb")
    pickle.dump(model2, pickle_out)
    pickle_out.close()
```

In this research , we have implemented KNeighbourClassifier, Logistic Regression Multinomial Naive Bayes and Linear SVC.

1. KNeighborsClassifier has the highest accuracy score on both the training (0.993625) and test (0.793552) sets among all the algorithms evaluated.
2. For the training set, LinearSVC has the highest recall score (0.954750), while KNeighborsClassifier has the highest precision score (0.994235) and F1-score (0.991853).
3. For the test set, KNeighborsClassifier again has the highest precision score (0.942271) and F1-score (0.783770), while LogisticRegression has the highest recall score (0.762264).
4. There is a noticeable drop in performance from the training to the test set for all algorithms, indicating some degree of overfitting on the training data. This drop is most pronounced for KNeighborsClassifier, with its accuracy decreasing from 0.993625 to 0.793552.
5. MultinomialNB has the lowest scores across most metrics on both the training and test sets, suggesting it may not be the best algorithm for this particular

classification task.

6. The choice of the best algorithm depends on the specific requirements of the problem. If overall accuracy is the priority, `KNeighborsClassifier` appears to be the best choice. However, if recall (identifying positive instances) is more important, `LogisticRegression` may be preferred for the test set, despite having a lower overall accuracy.

	Algorithm	Accuracy Score : Train	Recall Score : Train	F1-Score : :Train	Precision Score : :Train	Accuracy Score : : Test	Recall Score : Test	F1-Score : Test	Precision Score : : Test
0	KNeighborsClassifier	0.993625	0.989482	0.991853	0.994235	0.793552	0.667410	0.783770	0.949271
1	LogisticRegression	0.964187	0.956683	0.953994	0.951321	0.849038	0.762264	0.824009	0.896639
2	MultinomialNB	0.893938	0.873767	0.862357	0.851241	0.798050	0.733739	0.749224	0.765377
3	LinearSVC	0.963938	0.954750	0.953755	0.952762	0.845289	0.755876	0.820528	0.897273

Chapter 6

Conclusion

In this research after implementing all the decided algorithm it is crystal clear that, KNeighborsClassifier has the highest overall accuracy on both the training and test sets, making it the top-performing algorithm in terms of this metric. However, it also exhibits the largest performance drop from the training to the test set, suggesting potential overfitting issues. For the training set, LinearSVC has the highest recall score, while KNeighborsClassifier has the highest precision and F1-score. This indicates that LinearSVC is better at identifying positive instances during training, but KNeighborsClassifier strikes a better balance between precision and recall. On the test set, KNeighborsClassifier maintains the highest precision and F1-score, but LogisticRegression has the highest recall score. This suggests that LogisticRegression may be a better choice if the priority is to identify as many positive instances as possible, even at the cost of some false positives. MultinomialNB consistently underperforms compared to the other algorithms across most metrics, indicating that it may not be well-suited for this particular classification task. The choice of the best algorithm ultimately depends on the specific requirements and priorities of the problem at hand. If overall accuracy is the primary concern, KNeighborsClassifier may be the preferred choice. However, if recall is more important, LogisticRegression could be a better option for the test set, despite having a slightly lower overall accuracy. In conclusion, while KNeighborsClassifier emerges as the top performer in terms of overall accuracy, the results highlight the importance of considering multiple performance metrics and the trade-offs between them, as well as evaluating models on both training and test data to assess their generalization capabilities. The final selection of the algorithm should be guided by the specific needs and constraints of the classification problem at hand.

References

- [1]M. T. Ahmed, M. Rahman, S. Nur, A. Z. M. T. Islam, and D. Das, “Natural language processing and machine learning based cyberbullying detection for Bangla and Romanized Bangla texts,” TELKOMNIKA, vol. 20, no. 1, p. 89, 2021.
- [2]A. R. Pal, D. Saha, N. S. Dash, S. K. Naskar, and A. Pal, “A novel approach to word sense disambiguation in Bengali language using supervised methodology,” Sadhana, vol. 44, no. 8, 2019.
- [3]A. R. Pal and D. Saha, “Word Sense Disambiguation in Bengali language using unsupervised methodology with modifications,” Sadhana, vol. 44, no. 7, 2019.
- [4]M. F. Ahmed, Z. Mahmud, Z. T. Biash, A. A. N. Ryen, A. Hossain, and F. B. Ashraf, “Cyberbullying detection using deep neural network from social media comments in Bangla language,” arXiv [cs.CL], 2021.
- [5]M. Kaysar, “Word sense disambiguation for Bangla words using apriori algorithm,” 2018.
- [6]P. K. Saha, A. Das Mou, and T. Mittra, “A Bangla word sense disambiguation technique using minimum edit distance algorithm and cosine distance,” in 2019 23rd International Computer Science and Engineering Conference (ICSEC), 2019, pp. 1–6.
- [7]A. R. Pal, D. Saha, N. S. Dash, and A. Pal, “Word sense disambiguation in Bangla language using supervised methodology with necessary modifications,” J. Inst. Eng. (India) Ser. B, vol. 99, no. 5, pp. 519–526, 2018.
- [8]A. R. Pal and D. Saha, “Word sense disambiguation in Bengali: An unsupervised approach,” in 2017 Second International Conference on Electrical, Computer and Communication Technologies (ICECCT), 2017, pp. 1–5.
- [9]S. Nazah, M. M. Hoque, and M. R. Hossain, “Word sense disambiguation of Bangla sentences using statistical approach,” in 2017 3rd International Conference on Electrical Information and Communication Technology (EICT), 2017, pp. 1–6.
- [10]S. Xu, Y. Li, and Z. Wang, “Bayesian Multinomial Naïve Bayes Classifier to Text Classification,” in Lecture Notes in Electrical Engineering, Singapore: Springer Singapore, 2017, pp. 347–352.

- [11]V. Mitra, C.-J. Wang, and S. Banerjee, "Text classification: A least square support vector machine approach," *Appl. Soft Comput.*, vol. 7, no. 3, pp. 908–914, 2007.
- [12]C.-C. Chang and C.-J. Lin, "LIBSVM: A library for support vector machines," *ACM Trans. Intell. Syst. Technol.*, vol. 2, no. 3, pp. 1–27, 2011.
- [13]T. Joachims, "Text categorization with Support Vector Machines: Learning with many relevant features," in *Machine Learning: ECML-98*, Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, pp. 137–142.
- [14]S. Maji, A. C. Berg, and J. Malik, "Classification using intersection kernel support vector machines is efficient," in *2008 IEEE Conference on Computer Vision and Pattern Recognition*, 2008, pp. 1–8.
- [15]B. Scholkopf and A. J. Smola, *Learning with kernels: Support vector machines, regularization, optimization, and beyond*. London, England: MIT Press, 2001.
- [16]R. E. Wright, "Logistic regression," in *Reading and understanding multivariate statistics*, (pp, vol. 373, L. G. Grimm, Ed. Washington, DC, US: American Psychological Association, ix, 1995, pp. 217–244.
- [17]T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [18]Y.-Y. Song and Y. Lu, "Decision tree methods: applications for classification and prediction," *Shanghai Arch. Psychiatry*, vol. 27, no. 2, pp. 130–135, 2015.
- [19]L. Breiman, *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [20]G. Forman, I. Guyon, and A. Elisseeff, "An extensive empirical study of feature selection metrics for text classification," *Jmlr.org*, 2003. [Online]. Available: https://www.jmlr.org/papers/volume3/forman03a/forman03a_full.pdf. [Accessed: 04-May-2023].
- [21]N. S. Altman, "An introduction to kernel and nearest-neighbor nonparametric regression," *Am. Stat.*, vol. 46, no. 3, pp. 175–185, 1992.
- [22]T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [23]S. Albawi, T. A. Mohammed, and S. Al-Zawi, "Understanding of a convolutional neural network," in *2017 International Conference on Engineering and Technology (ICET)*, 2017, pp. 1–6.

- [24]Y. Yu, X. Si, C. Hu, and J. Zhang, “A review of recurrent neural networks: LSTM cells and network architectures,” *Neural Comput.*, vol. 31, no. 7, pp. 1235–1270, 2019.
- [25]N. B. Harikrishnan, “Confusion matrix, accuracy, precision, recall, F1 score,” *Analytics Vidhya*, 10-Dec-2019. [Online]. Available: <https://medium.com/analytics-vidhya/confusion-matrix-accuracy-precision-recall-f1-score-ade299cf63cd>. [Accessed: 05-May-2023].
- [26]P. F. Brown, S. A. D. Pietra, V. J. D. Pietra, and R. L. Mercer, “Word-sense disambiguation using statistical methods,” in *Proceedings of the 29th annual meeting on Association for Computational Linguistics -*, 1991.
- [27]P. Resnik and D. Yarowsky, “A perspective on word sense disambiguation methods and their evaluation,” *Aclanthology.org*. [Online]. Available: <https://aclanthology.org/W97-0213.pdf>. [Accessed: 07-May-2023].
- [28]D. Yarowsky, “Unsupervised word sense disambiguation rivaling supervised methods,” in *Proceedings of the 33rd annual meeting on Association for Computational Linguistics -*, 1995.
- [29]G. Escudero, L. Màrquez, and G. Rigau, “Boosting applied to word sense disambiguation,” in *Machine Learning: ECML 2000*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 129–141.
- [30]T. A. Atoorse, T. Particularetter, and P. T. A. Compu-(Eu-, “Adam.Kilgarri @itri.bton.ac.uk,” *Sketchengine.eu*. [Online]. Available: <https://www.sketchengine.eu/wp-content/uploads/2015/04/1998-Senseval-Euralex.pdf>. [Accessed: 07-May-2023].
- [31]P. Edmonds and A. Kilgarrieff, “Introduction to the special issue on evaluating word sense disambiguation systems,” *Nat. Lang. Eng.*, vol. 8, no. 4, pp. 279–291, 2002.
- [32]Y. S. Chan, H. T. Ng, and D. Chiang, “Word sense disambiguation improves statistical machine translation,” *Aclanthology.org*. [Online]. Available: <https://aclanthology.org/P07-1005.pdf>. [Accessed: 07-May-2023].
- [33]Z.-Y. Niu, D.-H. Ji, and C.-L. Tan, “I2R: Three systems for word sense discrimination, Chinese word sense disambiguation, and English word sense disambiguation,” *Aclanthology.org*. [Online]. Available: <https://aclanthology.org/S07-1037.pdf>. [Accessed: 07-May-2023].

- [34]D. Yuret, “KU: Word sense disambiguation by substitution,” Aclanthology.org. [Online]. Available: <https://aclanthology.org/S07-1044.pdf>. [Accessed: 07-May-2023].
- [35]R. Pawar and R. R. Raje, “Multilingual Cyberbullying Detection System,” in 2019 IEEE International Conference on Electro Information Technology (EIT), 2019, pp. 040–044.
- [36]M. Dadvar, F. De Jong, R. Ordelman, and D. Trieschnigg, “Improved cyberbullying detection using gender information,” Utwente.nl. [Online]. Available: https://ris.utwente.nl/ws/files/5512243/DIR12_reviewed04.pdf. [Accessed: 07-May-2023].
- [37]C. Iwendi, G. Srivastava, S. Khan, and P. K. R. Maddikunta, “Cyberbullying detection solutions based on deep learning architectures,” Multimed. Syst., 2020.
- [38]B. S. Nandhini and J. I. Sheeba, “Cyberbullying detection and classification using information retrieval algorithm,” in Proceedings of the 2015 International Conference on Advanced Research in Computer Science Engineering & Technology (ICARCSET 2015), 2015.